ФГБОУ ВО «Пензенский государственный университет»
Факультет вычислительной техники
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Бинарное дерево поиска»

Выполнили студенты группы 25ВВВ3:
Махотин Д.А.
Лысов И.В.
Приняли:
к.т.н, доцент Юрова О.В.
к.т.н., доцент Деев М.В.

Пенза 2026

Цель: Изучить принцип работы бинарного дерева поиска и реализовать основные операции с его элементами: добавление, поиск и подсчёт вхождений заданного значения.

Ход работы:

1. Добавлена проверка на наличие одинаковых элементов, поэтому повторяющиеся значения в дерево не добавляются.

```
if (data == r->data)
{
    printf("Число %d уже есть в дереве, повторно не добавлено\n", data);
    return root;
}

if (data < r->data)
    CreateTree(r, r->left, data);
else
    CreateTree(r, r->right, data);

return root;
```

2. Реализована функция поиска заданного числа в дереве.

```
struct Node* search(struct Node* r, int data)
{
    if (r == NULL) return NULL;
    if (data == r->data) return r;
    if (data < r->data) return search(r->left, data);
    return search(r->right, data);
}
```

3. Реализована функция подсчёта количества вхождений заданного значения.

```
int count_value(struct Node* r, int data)
{
    if (r == NULL) return 0;
    if (data < r->data) return count_value(r->left, data);
    if (data > r->data) return count_value(r->right, data);
    return 1;
}
```

4. Добавлен вывод дерева на экран и освобождение выделенной памяти.

```
void print_tree(struct Node* r, int l)
{
    if (r == NULL)
    {
        return;
    }

    print_tree(r->right, l + 1);
    for (int i = 0; i < l; i++)
    {
        printf("    ");
    }
    printf("%d\n", r->data);
    print_tree(r->left, l + 1);
}
```

```c
int main()
{
    setlocale(LC_ALL, "");

    int D, choice;

    printf("-1 - окончание построения дерева\n");
    while (1)
    {
        D = read_int("Введите число: ");
        if (D == -1)
        {
            printf("Построение дерева окончено\n\n");
            break;
        }
        root = CreateTree(root, root, D);
    }

    printf("Дерево (корень слева):\n");
    print_tree(root, 0);


    do
    {
        printf("\n1 - найти число, 2 - посчитать вхождения, 0 - выход\n");
        choice = read_int("Выбор: ");

        if (choice == 1)
        {
            D = read_int("Какое число искать: ");
            if (search(root, D) != NULL) printf("Число %d найдено в дереве\n", D);
            else printf("Числа %d в дереве нет\n", D);
        }
        else if (choice == 2)
        {
            D = read_int("Вхождения какого числа посчитать: ");
            printf("Число %d встречается %d раз(а)\n", D, count_value(root, D));
        }
    } while (choice != 0);

    free_tree(root);
    return 0;
```

Сложность поиска

Поиск в бинарном дереве начинается с корня. На каждом шаге мы сравниваем искомое число с текущим узлом и уходим в одно из двух поддеревьев, а второе целиком отбрасываем. Значит, количество шагов не больше высоты дерева, то есть длины самого длинного пути от корня до листа.

- Лучший случай. Искомое число лежит в корне: 1 сравнение, сложность O(1).
- Средний случай и сбалансированное дерево. Если числа вводились в «случайном» порядке, дерево получается ровным, а его высота около $\log_2(n)$. Каждый шаг отсекает примерно половину оставшихся элементов, поэтому сложность O(log n). Для миллиона элементов это порядка 20 шагов.
- Худший случай. Если числа вводить уже отсортированными (1, 2, 3, 4, …), каждое следующее число идёт в одну и ту же сторону, и дерево вырождается в «цепочку», по сути в связный список. Тогда высота равна n, и поиск требует прохода по всем элементам: сложность O(n).

Вывод: В ходе лабораторной работы было создано бинарное дерево поиска и реализованы операции добавления элементов, поиска значения и подсчёта его вхождений. Также была реализована проверка на наличие дубликатов. В результате были изучены основные принципы работы с бинарными деревьями поиска.